class07: Introduction to machine learning for Bioinformatics

Hubert Phan

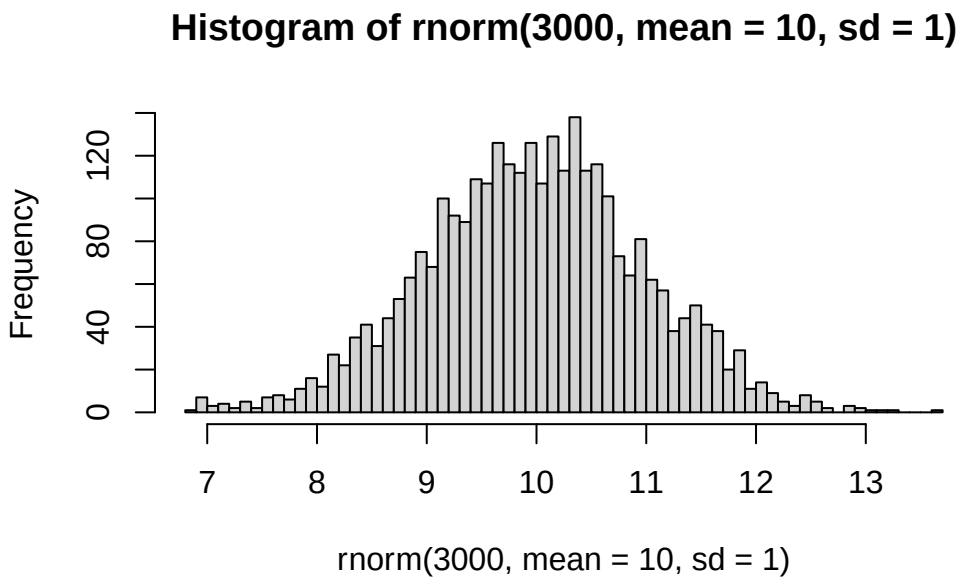
Table of contents

Background

Today we will begin our explortion of important machine learning methods with a focus on **clustering** and **dimensionallity reduction**.

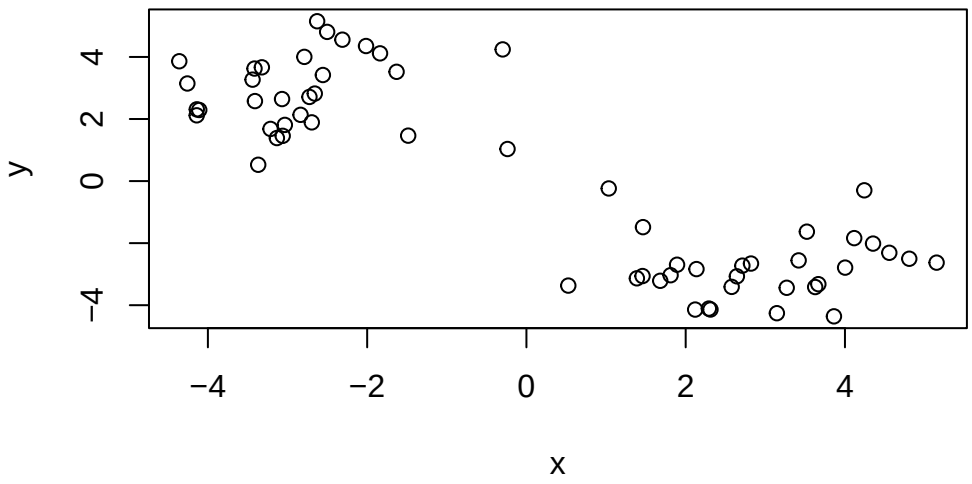
To start testing these methods let's make up some sample data to cluster where we know what the answer should be.

```
hist(rnorm(3000, mean = 10, sd = 1), breaks = 50)
```



Q. Can you generate 30 numbers centered at +3 and 30 at -3 taken at random from a normal distribution?

```
tmp1 <- rnorm(30, mean = 3)
tmp2 <- rnorm(30, mean = -3)
#combine the two into one vector
data <- c(tmp1, tmp2)
#combine by columns
x <- cbind(x = data, y = rev(data))

plot(x)
```



K-means clustering

The main function in "base R" for k-means clustering is `kmeans()`. The function takes two main arguments: the data to be clustered and the number of clusters to create.

```
#run kmeans on x, with 2 clusters
km.out <- kmeans(x, centers = 2)
km.out
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1  2.886596 -2.822494
2 -2.822494  2.886596

Clustering vector:
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

Within cluster sum of squares by cluster:
[1] 72.30638 72.30638
```

```
 (between_SS / total_SS =  87.1 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

Q. What component of your kmeans result object has the cluster centers?

```
km.out$centers #centroid pointss
```

```
          x         y
1  2.886596 -2.822494
2 -2.822494  2.886596
```

Q. What component of your kmeans result object has the cluster size(ie. how many points are in each cluster)?

```
km.out$size #number of points in each cluster
```

```
[1] 30 30
```

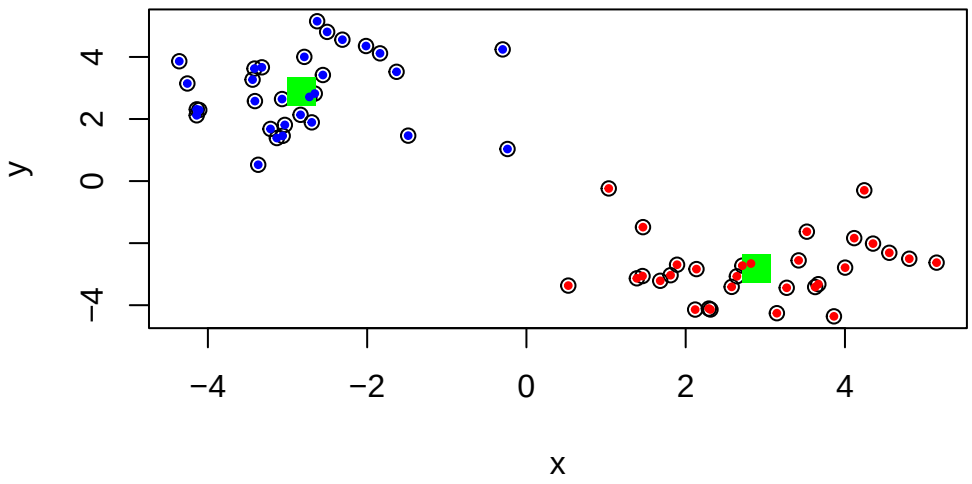
Q. What component of your kmeans result object has the cluster member vector(ie. the main clustering result)?

```
km.out$cluster #cluster membership vector
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

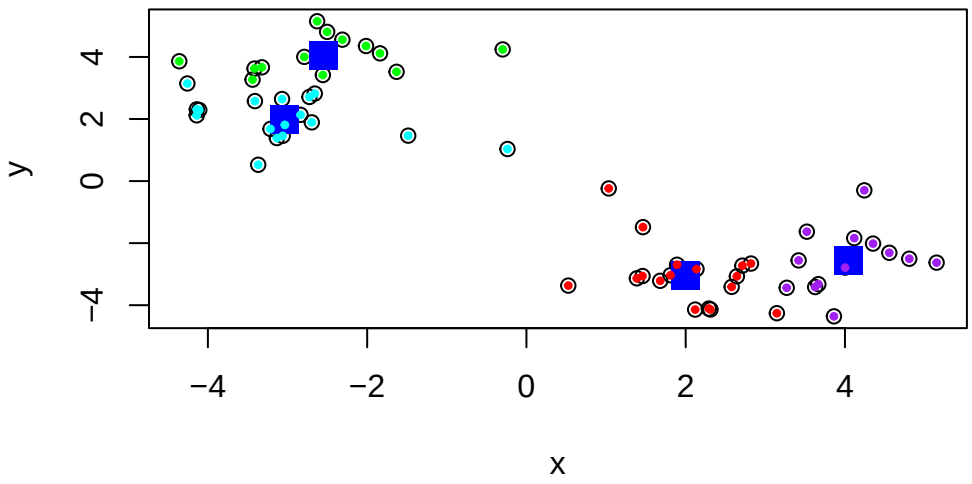
Q. Plot the results of clustering (ie. out data colored by the clustering result)

```
plot(x)
my_colors <- c("red", "blue")
points(km.out$centers, col = "green", pch = 15, cex = 2)


points(x, col = my_colors[km.out$cluster] , pch = 20, cex = 0.7)
```



Q. Run kmeans again and cluster into 4 clusters and plot the results.

```
#kmeans with 4 centers
km.out2 <- kmeans(x, centers = 4)

plot(x)
my_colors2 <- c("red", "cyan", "green", "purple")
points(km.out2$centers, col = "blue", pch = 15, cex = 2)
points(x, col = my_colors2[km.out2$cluster] , pch = 20, cex = 0.7)
```



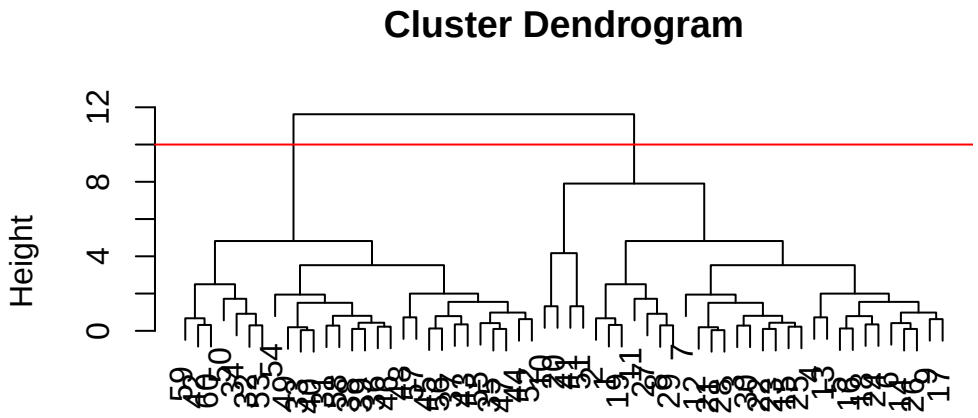
Key Point: K-means clustering always return the clustering that we ask for (this is the "K" or "centers" in K-means). It is up to the user to decide what K is appropriate for their data.

Hierarchical Clustering

The main functions in "base R" for hierarchical clustering are `dist()` and `hclust()`. The `dist()` function computes the distance matrix between all pairs of points in the data, while the `hclust()` function performs hierarchical clustering on the distance matrix.

One of the main differences with respect to the `kmeans()` function is that you can't just pass your input data directly to `hclust()`, you need to first compute the distance matrix with `dist()`.

```
#compute distance matrix
dist.x <- dist(x, method = "euclidean")
#perform hierarchical clustering
hc.out <- hclust(dist.x)
plot(hc.out)
abline(h = 10, col = "red")
```

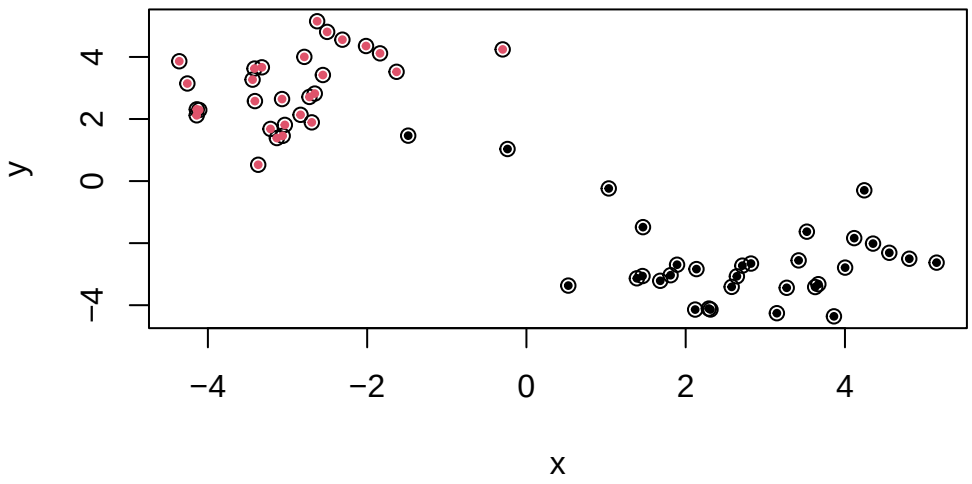



```
#We can cut the dendogram at a given height to yield a certain number of clusters
clusters <- cutree(hc.out, h = 10)
clusters
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 1 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2
```

Q. plot our data `x` colored by the clustering result from hierarchical clustering.

```
plot(x)
points(x, col = clusters , pch = 20, cex = 0.7)
```



Principal Component Analysis (PCA)

PCA is a popular dimensionallity reduction technique that transforms a dataset into a new coordinate system such that the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on.

PCA of UK food data

Data Import

Read in the UK food data from the following URL:

```
#load UK food data
url <- "https://tinyurl.com/UK-foods"
food_data_tmp <- read.csv(url)
```

Read data in properly. Fix row names assignment at import

```
#make column 1 the row names
food_data <- read.csv(url, row.names = 1)
head(food_data)
```

```
               England Wales Scotland N.Ireland
Cheese             105   103      103        66
Carcass_meat       245   227      242       267
Other_meat         685   803      750       586
Fish               147   160      122        93
Fats_and_oils      193   235      184       209
Sugars             156   175      147       139
```

> **Q1.** How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(food_data) #number of rows and columns
```

```
[1] 17  4
```

```
nrow(food_data) #number of rows
```

```
[1] 17
```

```
ncol(food_data) #number of columns
```

```
[1] 4
```

Checking your data

> **Q2.** Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I like the method where we fix the column to the row names as we read in the data. This prevents us from having to do an extra step after reading in the data. Furthermore, we dont accidentally delete any columns if we rerun the code.

Spotting major differences and trends

> **Q3:** Changing what optional argument in the above barplot() function results in the following plot?

Using `beside = FALSE` in the barplot function results in the following plot.

Modify the data to make it plottable with ggplot. We are changing from wide to long format using the `pivot_longer()` function from the `tidyverse` package.

```
library(tidyr)
```

```
Warning: package 'tidyr' was built under R version 4.5.2
```

```
# Convert data to long format for ggplot with `pivot_longer()`
x_long <- food_data |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

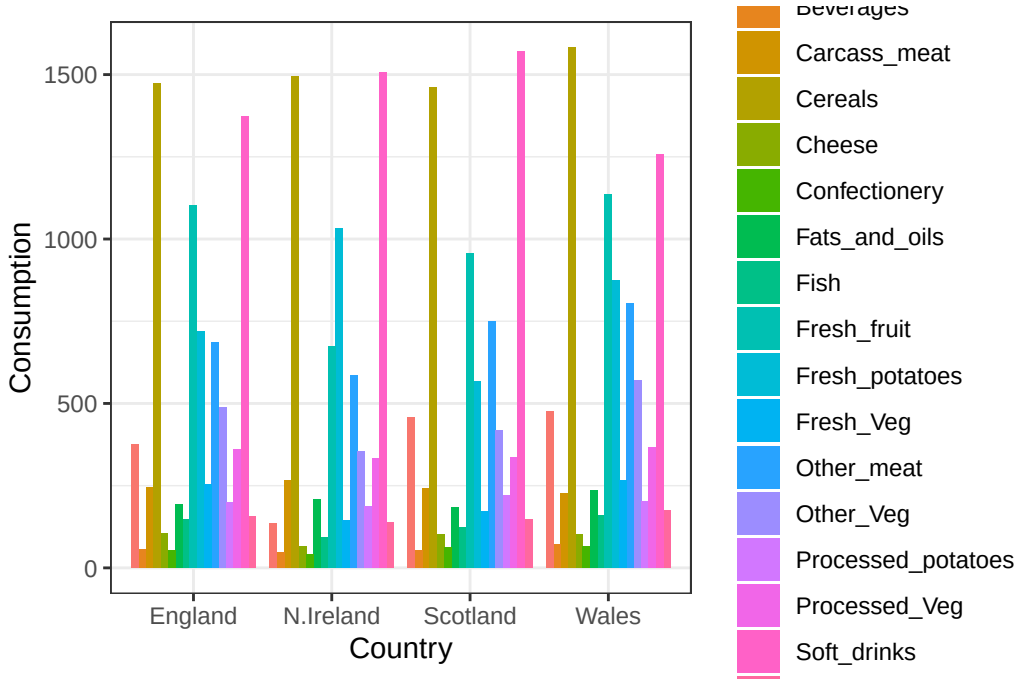
```
[1] 68  3
```

ggplot result:

```
# Create grouped bar plot
library(ggplot2)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

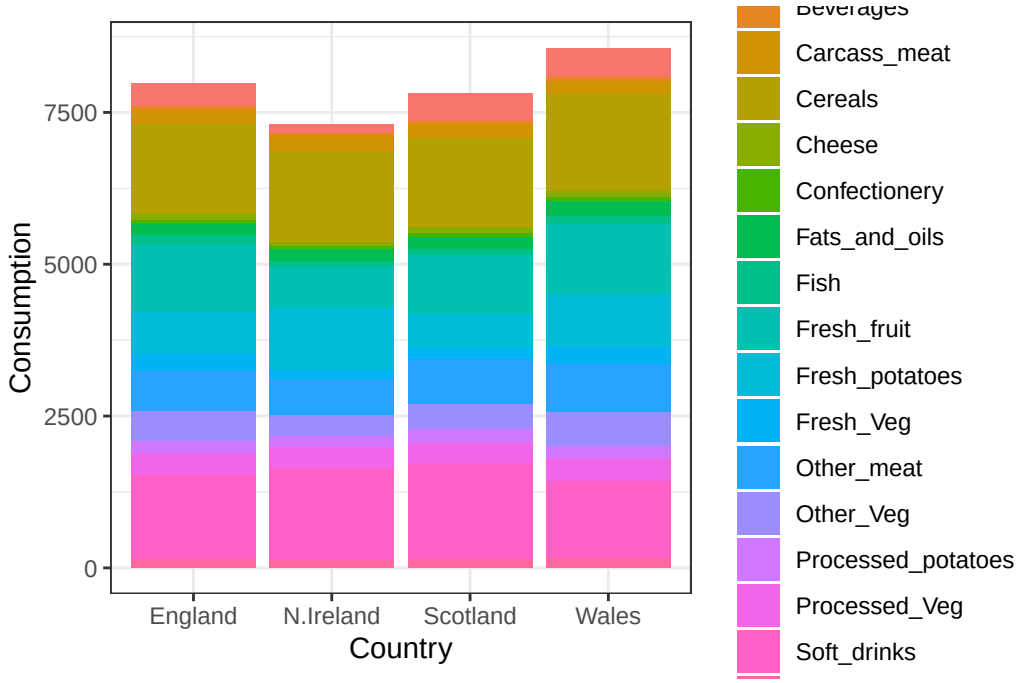
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

In geom_col, change to `position = "stack"` to get a stacked barplot.

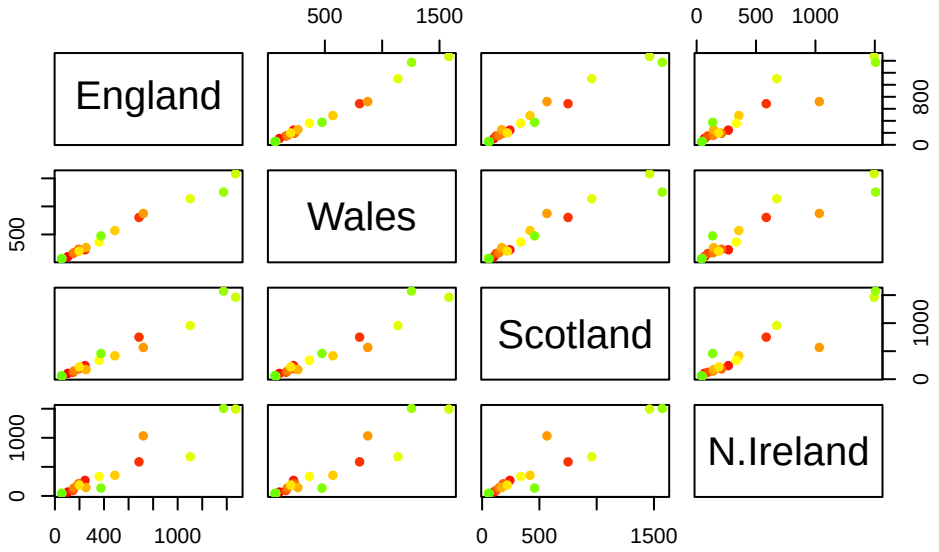
```
# Create stacked bar plot
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



Pairsplots and heatmaps

Q5: We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(food_data, col=rainbow(nrow(x)), pch=16)
```



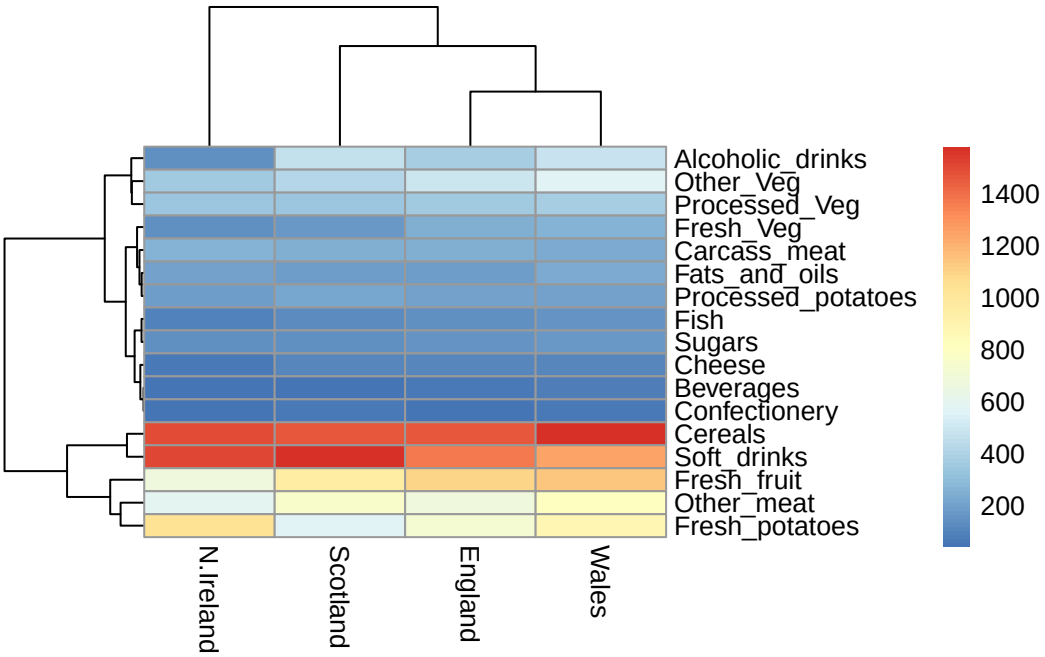
If a point is on the diagonal for a given plot, it means that the consumption of that food item is the same in both countries being compared.

Heatmap:

```
library(pheatmap)
```

```
Warning: package 'pheatmap' was built under R version 4.5.2
```

```
pheatmap( as.matrix(food_data) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

From the plots we see that England, Wales, and Scotland cluster together, while Northern Ireland forms its own cluster. We see that Northern Ireland has higher consumption of fresh potatoes and soft drinks compared to the rest of the UK. Of all of the plots, `pairs()` was the most useful. It did take a bit of work to interpret and will be even more difficult with larger datasets.

PCA to the rescue!

The main base R function for PCA is `prcomp()`. The function takes the data to be analyzed as its main argument.

```
#perform PCA on food data
pca <- prcomp(t(food_data))
summary(pca)
```

```
Importance of components:
                            PC1       PC2       PC3        PC4
```

```
Standard deviation     324.1502 212.7478 73.87622 3.176e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

Q. How much variance is captured in the first PC?

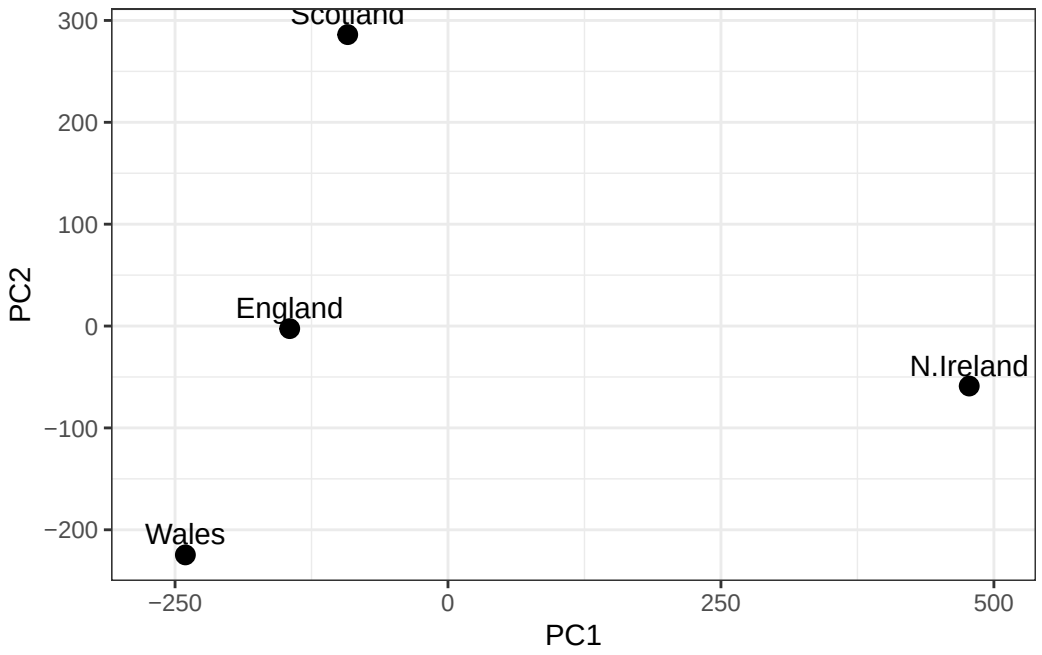
The first principal component (PC1) captures 67.4% of the variance.

Q. How many PCs do I need to capture at least 90% of the variance?

I need 2 principal components (PCs) to capture at least 90% of the variance.

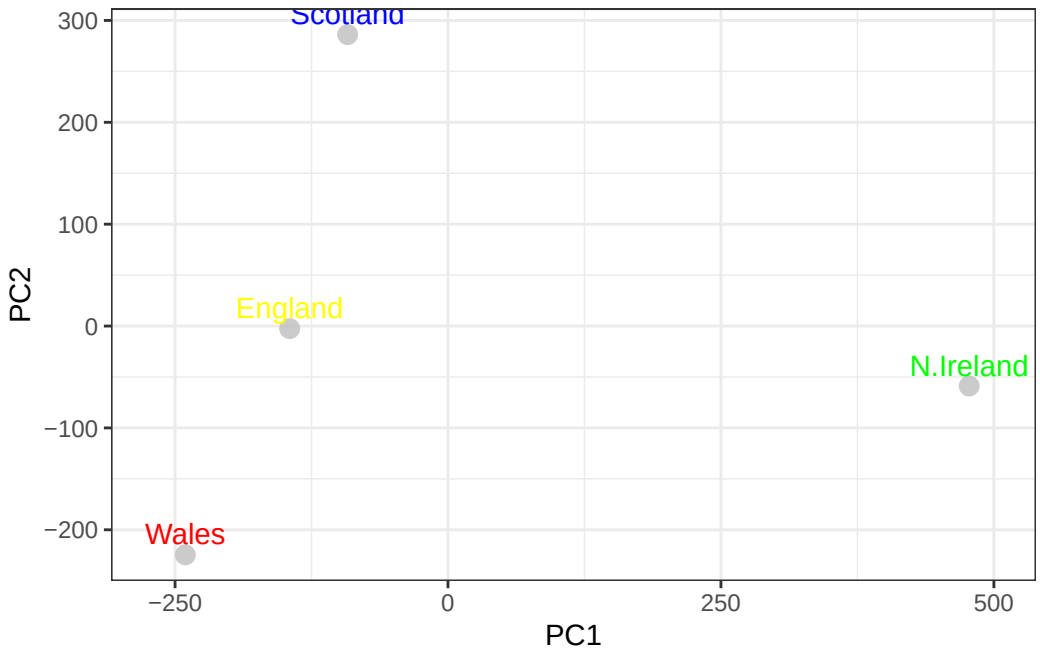
Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

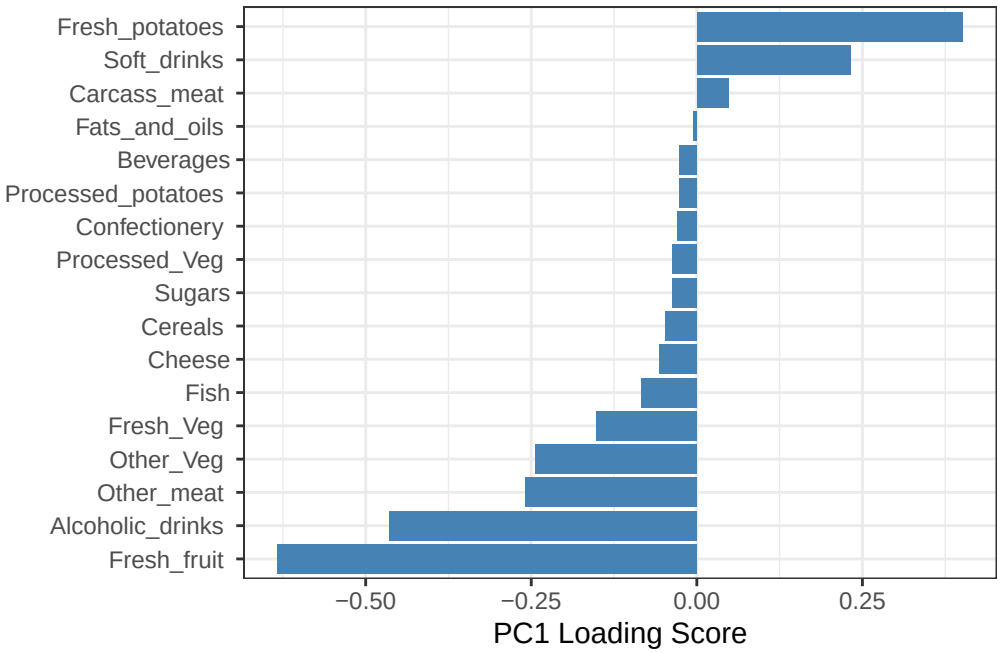
```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

my_colors3 <- c("England" = "yellow", "Wales" = "red", "Scotland" = "blue", "N.Ireland" = "gr
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3, col = "gray", alpha = 0.8) +
  geom_text(aes(color = rownames(pca$x)), vjust = -0.5) +
  scale_color_manual(values = my_colors3) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw() +
  theme(legend.position = "none")
```



Digging deeper (variable loadings)

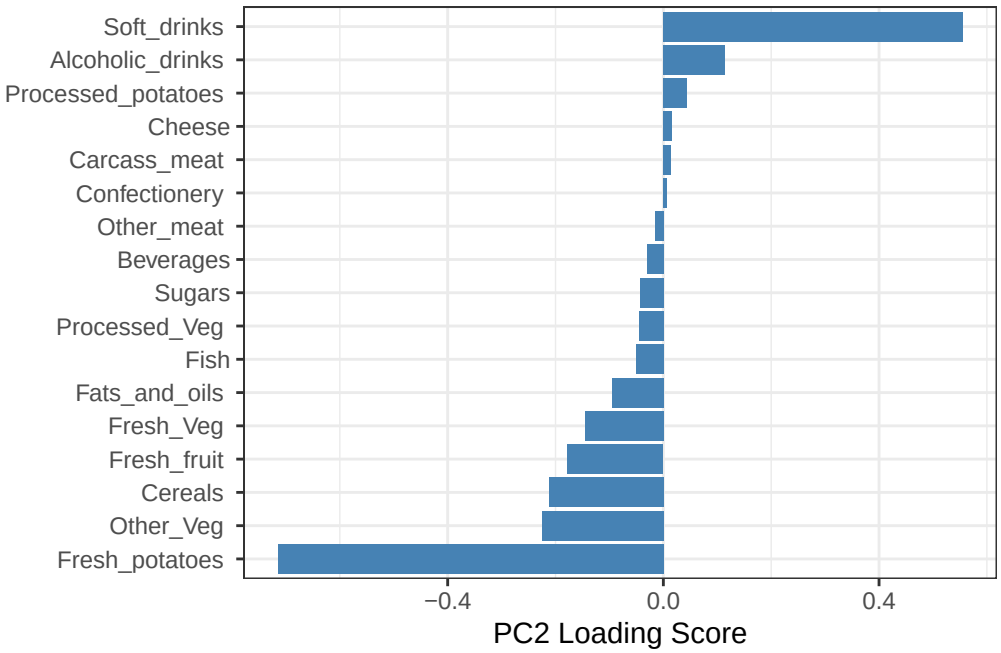
How do the original variables (countries) contribute to the new PCs? We can look at the loadings of each variable on each PC.

```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominantely and what does PC2 maninly tell us about?

```
## Lets focus on PC2
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The two food groups that feature predominantly are softdrinks and fresh potatoes. In PC2, positive scores associate with Scotland, near zero scores associate with England, and negative scores associate with Wales and Northern Ireland. PC2 contrasts drink heavy diets vs. staple/produce heavy diets. From PC2, we can assign Scotland to soft drinks and Wales to Fresh potatoes.